Claude Certified Architect – Foundations (CCA-F)

notes from the freeCodeCamp.org video · course by Andrew Brown (ExamPro) · ~12.6 hrs

1. What is this cert?

- Anthropic's certification — the broad concepts of building agents + Claude's models, coding tools & SDKs
- Focus: org workloads + **MCP** (Anthropic created MCP → know it well)
- "CCA-F" = no official exam code, it's just what everyone calls it
- For: building agentic workflows · adopting Claude org-wide · **Claude Partner Network** members

Exam facts

60 questions (MCQ)	**720** to pass = 72%
120 minutes	**5** domains

- can get ~17 wrong
- usual exams pass at 70% — this is **72%**
- seat time 150 min
- delivered via **Skilljar**
- validity period: unknown

Exam tip! Questions are very verbose scenarios (pro-level architect style) — parsing is the hard part. ~2 min each, read carefully.

The 5 domains

1. Agentic architecture & orchestration ← *lots of new stuff*
2. Tool design & MCP integration
3. Claude Code configuration & workflows
4. Prompt engineering & structured output
5. Context management & reliability ← *new + very valuable*

⚠ Availability: at recording, Claude Partner Network only (has org-size requirements).

2. How to prepare

Recommended path



- ~30% of CCA-F content comes from Claude Code Essentials → ≈ half of Domain 2 + all of Domain 3
- Domains 1 & 5 are mostly **completely new concepts** for most people
- "Not an easy course" — you need developer experience
- Agent SDK / Claude Code setup lives in the Essentials course

Time budget

- beginner: **~30 hrs** (maybe more — concepts are new even for the instructor)
- experienced: course is ~12 hrs; even doubled, plan time accordingly
- split 50% lectures + labs / 50% practice exams
- ~1 hr a day · don't cram in one day (burnout) · don't drag it out (you'll forget)

Approach

- Course **walks the exam guide in order**, implementation-focused
- Anthropic's free courses = concept-based, not enough on their own
- Do the **hands-on labs** (all code provided) — LLMs are chaotic; use videos as a reference

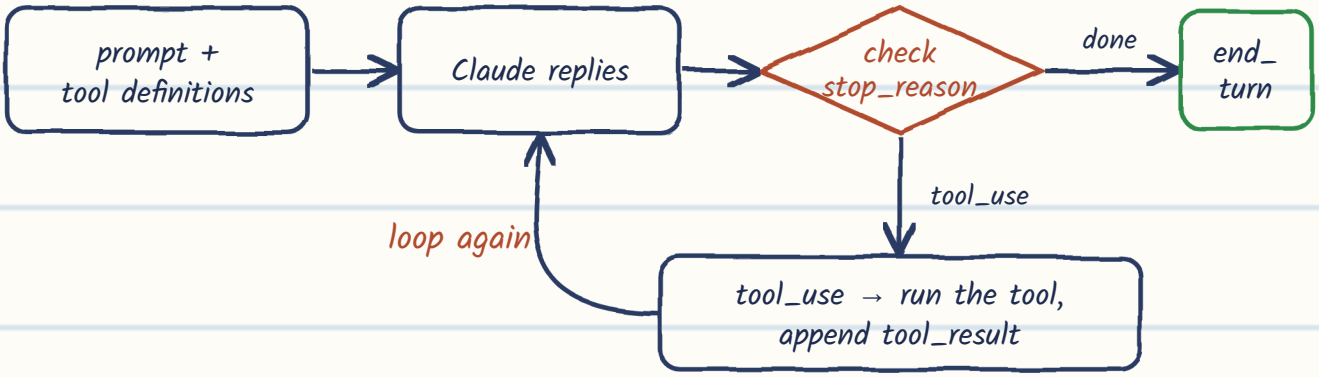
Big lesson: verify, don't trust. *The instructor found the docs, the exam guide and the AI were sometimes wrong. Only by building it did they find "the real truth." E.g. Claude didn't know Task → Agent rename; agent definition fields differed between Python & TypeScript SDKs.*

Setup chapter (quick notes)

- install Claude Code, then the Agent SDK via ***pip install***
- subscription login = most cost-effective for learning; production would use an API key
- Plan mode: ***Shift+Tab*** twice · built a small helper library to make SDK output human-readable

3. The agentic loop & stop_reason

stop_reason = why Claude stopped executing its loop. Two values to know:



- ***tool_use*** — Claude wants a tool; your code runs it and appends the result back into the messages, then calls again
- ***end_turn*** — Claude decided it's done → returns the final result
- What's in *messages* ≠ what's output, so you don't see stop_reason inside the messages

Demo: "what's the weather in Winnipeg?" → "let me check the weather for you" → tool ***get_weather****, input "Winnipeg" (extracted from the user) → then end_turn.*

The instructor hand-typed one tool's **input schema** (properties, type: string, required) once to show the shape — looks a lot like a Pydantic/JSON schema.

Models in the loop (Claude Code)

- uses the same model for all phases of the agentic loop
- **Opus** — slow, very smart, complex reasoning · **Sonnet** — the "Goldilocks", daily coding · **Haiku** — fast, simple tasks
- **Sonnet 1M** — 1M-token context for long sessions · **opusplan** — Opus during plan mode, Sonnet for execution
- API = you see token usage; subscription = hard to gauge usage per model

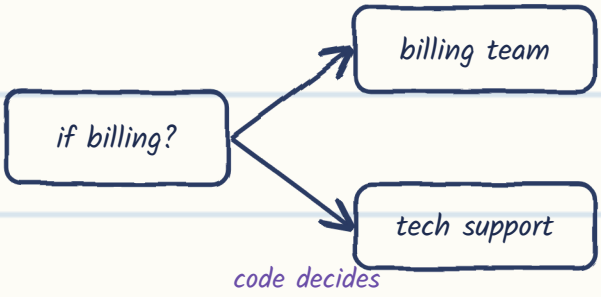
4. Decision trees vs model-driven

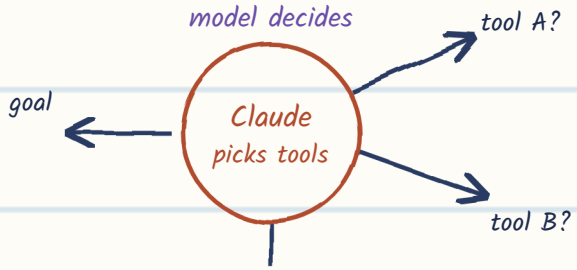
Preconfigured decision trees

- you **hard-code** the logic; the code decides what happens next
- simple: if / else
- complex: a **state machine** (initial state → transitions)
- Anthropic may call it a "*tool sequence*"
- early agents were often built as state machines

Model-driven decisions

- = an actual agentic model
- hand Claude **tools + a goal**
- Claude figures out the sequence at runtime from context
- you don't tell it what to call or when
- tools don't even need to be listed in the system prompt



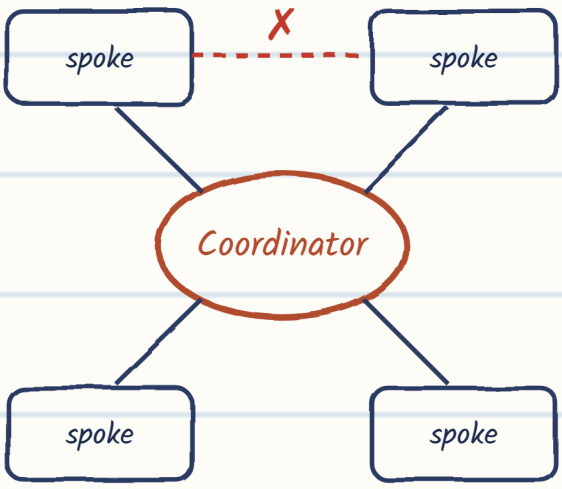


Demos from the video

- **Routing:** support messages routed — e.g. "route to the billing team, pull account record & check payment status"; "my app keeps crashing after latest update" → a different route
- **Ordering:** "I need five units of widget A. Check if they're available. Place the order" → Claude chains the inventory check and order tools itself

Remember: *decision tree = deterministic/predictable, code-controlled. Model-driven = flexible, Claude-controlled. Many real systems mix both (see the escalation state machine at the end!).*

5. Hub-and-spoke architecture



red ✗ = spokes never talk directly!

- one coordinator agent sits at the center
- all sub-agents (spokes) talk **only** to the coordinator
- sub-agents never have direct lines to each other
- likely one of the ways Claude Code itself works with sub-agents → "really useful, apply it immediately"
- you'll see the word **coordinator** a LOT

Hiring-screener demo

- spokes answer narrow questions; coordinator makes the **integrated judgment** on the hire
- keyword-scanner spoke prompt: "check whether required skills appear explicitly… Be literal. Do not infer or extrapolate."
- screening spoke: specialist analyst, answer in 2-3 focused sentences, concrete & specific
- idea: pass only the info a spoke needs (saves tokens) · test you can't pollute context between agents

Narrow decomposition = gaps!

EV-market research example: coordinator split into "battery tech" + "major manufacturers".

Each sub-agent sees only **its own isolated context** → none can flag what's missing.

missed: charging infrastructure, govt policies & subsidies, second-hand market, consumer sentiment, supply chain (lithium/cobalt), grid capacity

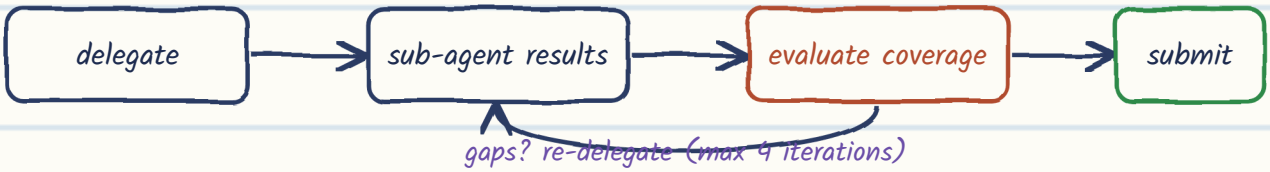
Tip: be very specific about the task so the decomposition fully covers what you expect.

6. Coordinator techniques

Dynamic selection (routing)

- running *every* spoke in sequence for every request = consuming as much as you can
- tell the coordinator to think about the routing it needs; give it routing ideas for certain circumstances
- "only invoke it if it makes sense"

Refinement loop



- before: everything one-shot → evaluation → over
- now: feed results back & refine until happy; the coordinator prompt allowed up to 4 refinement iterations
- prompt includes evaluation of coverage + **when to submit**

Error handling with multiple agents

- demo: sub-agent → coordinator for checking inventory, placing an order, notifications; goal = show **sub-agent failure recovery**
- loop safety: a **max-steps cap of 10** (double the ~5 expected steps) gives room to retry and catches runaway loops (= same idea as max iterations)
- results can come back as **partials** (e.g. a "maybe" recommendation with different coverage)

Refactoring the coordinator

- loop had grown into a big if/elif block → a **state machine** could improve it
- cleaned up constants, loaded prompts from template files, planning to port to the Agent SDK

Instructor's warning: *"just because we're doing it doesn't mean it's great." Really evaluating a design needs logs, examples & comparisons of the alternatives.*

7. The Agent tool & parallel delegation

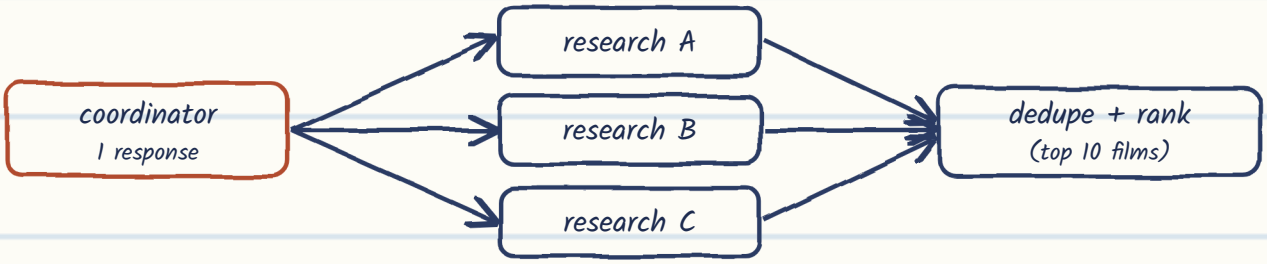
Agent tool (formerly the "Task" tool!)

- spins up a sub-agent with its own isolated context, its own **system prompt** and its own **set of tools**
- Task → Agent was just a **rename**; you'll still see "task" in parts of the code
- a sub-agent is not automatically parallel — depends on scheduling & prompting; generally it **blocks the parent** until it's done
- → another sign Claude Code uses hub-and-spoke internally
- JS tool names are lowercase_underscore; Python ones are title-cased

Agent definitions — watch the SDK gaps

The instructor made this video **4 times**: Claude's list of fields couldn't be confirmed in the docs, and testing showed fields that didn't exist. Some fields are TypeScript-only, some Python-only → the SDKs are not 1-to-1.

Parallel delegation



- film-research demo: the key to parallelism was the prompt — "in a single response call research agents simultaneously… **do not wait**, delegate to all of them"
- dedupe agent rules: if the same film appears more than once, keep the best entry, don't drop films, return a JSON object with 10 films
- quality: record findings with a per-finding **quality score** so the coordinator doesn't "mentally remember" criteria (unreliable); one IMDb source alone may not justify similarity claims

⚠ bypass permissions was on in the generated code — instructor doesn't like that.

8. Gates, chaining & provenance

Enforcement gates (hooks)

- built with **Agent SDK hooks**: ***PreToolUse***, ***PostToolUse*** (and failure hooks) — intercept tool calls before/after they run
- demo support tools: look up order, attempt refund, clear fraud flag… when it must escalate, it **builds a structured handoff package** for a human
- hook logging was tricky — only the final prompt came through, info was missing (needs deeper digging)

Prompt chaining (= sequential pipeline)



- a series of agents/calls where each output feeds the next
- good for: document processing, content transformation, **ETL pipelines** — any task where the shape of the work is fixed regardless of content
- not new — in early short-context LLM days, chaining was the only way to work

The "raw findings" dilemma

- passing findings between agents as **one blob of text** → attributions get lost
- the synthesis agent can't tell which claim came from which source → by the final output the provenance is gone

  provenance = where something came from (like proving an antique's origin)
- example claims in the blob: "EV adoption grew 40% in 2024", "battery costs have fallen"…
- **Fix:** keep findings structured with their sources; when merging, annotate conflicting stats with source attributions

Exam lens: *gate = deterministic enforcement around tools (hooks); chaining = fixed pipelines; multi-agent synthesis = preserve claim → source mapping.*

9. Sessions & context

Session = functionally a conversation with the Claude Code agent. Typing ***claude*** starts a new one. Resuming a session = resuming a past conversation.

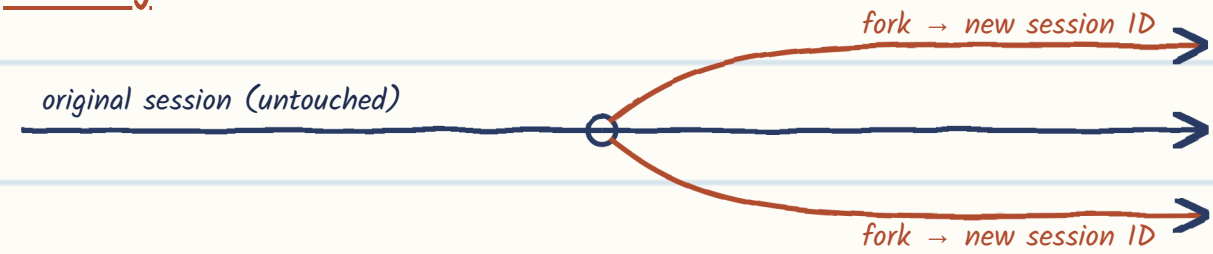
Two types

- **local** — started in your terminal/IDE
- **web** — started via remote control or Claude Code for web
- a web session could be resumed locally from the VS Code IDE (it may need the right folder)

Commands

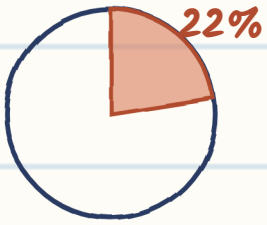
- ***--continue*** / ***--resume*** — pick up a past session
- ***--fork-session*** — branch it
- ***/compact*** — summarize history
- ***/clear*** — start clean

Forking



- resume a past session without affecting the original; each fork has its own UUID & saved history
- forks run in **parallel**, are **isolated**, and **don't share knowledge**
- Agent SDK: fork session was TypeScript-only at recording — not in Python. Claude itself kept insisting on an options flag → verify!

Autocompact buffer



Claude Code reserves ~22% of the context window so there is headroom to summarize history as limits approach. The context breakdown shows the system prompt, skills (small) and messages.

10. Settings & permission rules

settings.json — things you can set

- **attribution** — what commit/PR author text looks like (old *include co-authored-by* option is deprecated)
- **announcements** — messages cycled at startup (handy for companies)
- **env vars** applied to every session · **model** override
- **auto-memory directory** — custom path, not allowed in project settings · **plans directory**
- **additional directories** beyond project root (e.g. shared docs, coding guidelines)
- **default mode** (e.g. acceptEdits)
- **managed settings** lockdown flags — e.g. only managed permission rules, only managed MCP servers & hooks; hook settings (allowed hook URLs, env vars)

Permission rules

A tiered, tool-based access control system (instructor's joke name: "TBAC", like RBAC but around tools). Three lists:

allow	no need to ask	**ask**	confirm first	**deny**	sensitive files, dangerous cmds

Path patterns — 4 ways

//path	root of the **file system** (note: double slash!)
~/path	your **home** directory
/path	the project root (trips people up!)
./path or *path*	relative

- **WebFetch** rules just take a domain: ***domain:example.com*** in allow / ask / deny

*Gotcha from the lab: with Read (and Bash) denied on a file, Claude still read it with the **Grep** tool — Grep is a separate tool. Deny rules are per tool, so think about every tool that can reach the data.*

11. Permission modes & sandboxing

default	standard — prompts for permission on **first use of each tool**
acceptEdits	automatically accepts **file edits**
plan	can analyze but not modify files or execute commands
dontAsk	**auto-denies** tools unless pre-approved via permissions
bypassPermissions	skips **all** permission prompts — big warning

memory trick: default → asks · acceptEdits → edits OK · plan → read-only · dontAsk → silent "no" · bypass → anything goes

Sandboxing

Sandbox = a security mechanism separating running programs from your OS: tightly controlled resources — storage & memory scratch space, **limited network**, restricted inspection of the host, restricted input devices.

- enable with ***/sandbox***; needs OS packages — on **WSL 2** it asked for **bubblewrap**, **socat** and a **seccomp filter**
- lab: ***curl*** via Bash → denied "likely due to the sandbox restriction"
- network requests outside the sandbox → Claude **asks** whether to allow the connection
- reading ***~/.ssh*** config (outside the project) → blocked by file-system restrictions
- test idea: the sandbox catching something the deny rules missed



Safe pattern from the video: *if you want* ***--dangerously-skip-permissions****, do it inside a* ***virtual machine*** *(and/or the sandbox), not on your main machine.*

12. Built-in tools & tool_choice

Main built-in tools

Read	load files
Grep	search file **contents**
Glob	find files by **name pattern**
Edit	edit files (no Edit = can't)
Bash	run commands
LSP	code intelligence

some tools ask permission on first use (default mode); any tool can be denied.

tool_choice modes

- ***auto*** — Claude decides whether to use a tool; may return plain text → edge case that breaks structured output!
- ***any*** — must use **some** tool from the list
- ***tool*** — must use a specific named tool = "forced"



JSON Schema

- a **declarative language** for annotating & validating JSON structure, constraints and data types
- types: ***string***, ***integer***, ***boolean***, ***number***, ***array*** (items), nested ***object***, plus ***required***
- remember enums for the exam! e.g. ***unit***: string, enum [celsius, fahrenheit]; required: location + unit
- Claude reads the tool's input schema and knows what to generate; Pydantic is commonly used for structured output

Forced structured JSON via tools

Define the output shape as a tool's **input_schema** → force that tool with ***tool_choice*** → read the tool input as your JSON. (Lab hit a "name required" error — the forced tool_choice must name the tool.)

/debug — *outputs verbose info to a text file, and Claude asks you to describe the problem as it investigates. Useful for diagnostics.*

13. Reliability & large context

Multi-instance / multi-pass review



- the same agent reviewing its own work has that history → biased, thinks it did a great job
- never ask the generator to grade its own output in the same conversation — one generates, one reviews

Validation-retry feedback loops

- structured JSON comes back with syntax errors or other problems → **append the specific validation errors to the prompt & retry**
- Pydantic validates structure, but content can still be wrong → feed that back too
- *instructor: should be called "validate & remediate"*

Large-context problems

- **Progressive summarization risk:** condensing over multiple steps, the model doesn't know dates / % / numbers are important → makes them generic
- fix: extract key precise facts ("case facts") and carry them into later prompts
- **Verbose tool results:** a tool returning lots of JSON when you need a little (text-adventure demo) → pare it down
- chunking sections is another option

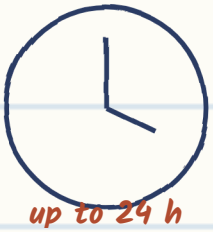
Large-codebase exploration (guide §5.4)

Context gets muddled as it grows — it forgets the tail or loses key info when it summarizes.

Use external memory: save findings to a file (e.g. ***findings.md***) as you crawl.

14. Batches, big codebases, CI/CD

Message batch processing

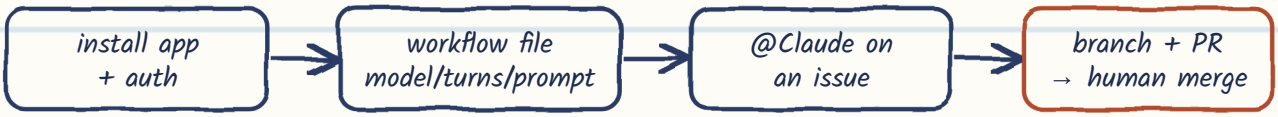


- not instant: up to 24 hours, no guaranteed SLA
- for the exam: know it exists and that it **saves money**
- each request gets a ***custom_id***; results come back keyed by it
- no interactive round trips → force a single tool call and extract its input directly
- script flow: submit → check (state file) → run & wait

Exploring a big real codebase

Demo on the **Doom** source: targeted questions like "what is the sequence of events & data transformations when the player receives damage?" (projectile collision → damage → pain-chance probability, invulnerability).

Claude Code in CI/CD (GitHub Actions)



- authenticated Claude Code + installed the GitHub Actions app; the action installs Claude Code, so it needed an **API token**
- workflow file lets you set **max turns**, **model** (run used Sonnet 4.6) and an **additional system prompt**
- every reply re-triggers the action (minutes each); hard to see which commands ran; not really a conversation
- got a PR only after asking; merge conflicts → define a branching strategy, keep code in small files

MCP: emphasized as part of Domain 2 (tool design & MCP integration); this short chapter in the video was mostly hands-on setup.

15. Scenario: support-bot escalation

Fictional "Exam Pro Training Inc." support agent (real ExamPro support is 100% human). Started with a **plan file** (business use case + context), v1 on Anthropic's **Agent SDK**.



Escalation triggers

- customer **explicitly asks** for a human
- AI **can't resolve** it after N turns
- AI decides it's **out of scope**
- some issues auto-escalate (GDPR, legal)

Implementation

- add an **escalate tool**
- track state + turn count in the session context
- map out state machines and tell the prompt what they are

Final exam checklist ✓

☐ stop_reason: ***tool_use*** vs ***end_turn*** · decision trees vs model-driven

☐ hub-and-spoke · dynamic selection · refinement loops · failure handling

☐ Agent tool (ex-Task), isolated context · parallel in a single response · hooks as gates · chaining · provenance

☐ sessions: resume / fork / compact / clear · 22% autocompact buffer

☐ permission modes (5) · allow/ask/deny · path patterns · sandbox

☐ tool_choice auto/any/tool · JSON Schema enums · forced JSON · validation-retry

☐ independent review · case facts · external memory · batches (24h, custom_id)

#1 tip: *do the labs and verify everything — docs, the exam guide and AI answers were all wrong at times.*